Experiment No. 7

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int**: unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool**: represents logical values (true or false).
- **address**: holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string**: store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure**: cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view**: can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:
    - public: available both inside and outside the contract.
    - private: only accessible within the same contract.
    - internal: accessible within the contract and its child contracts

Modifiers are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).

- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

\Arrays: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.

Mappings: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.

Structs: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.

Enums: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

storage: Data stored permanently on the blockchain. Examples: state variables.

memory: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.

calldata: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

Ether and Wei: Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.

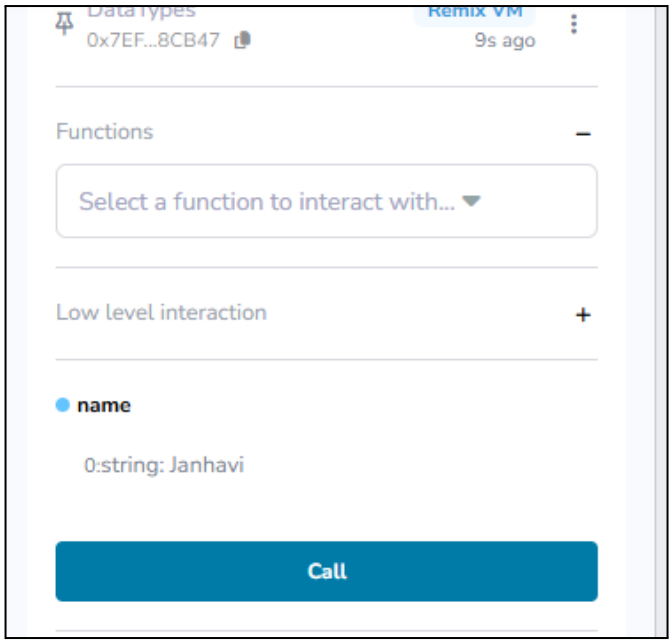
Gas and Gas Price: Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.

Sending Transactions: Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Implementation:

Tutorial no. 1 Introduction to Solidity and Smart Contracts

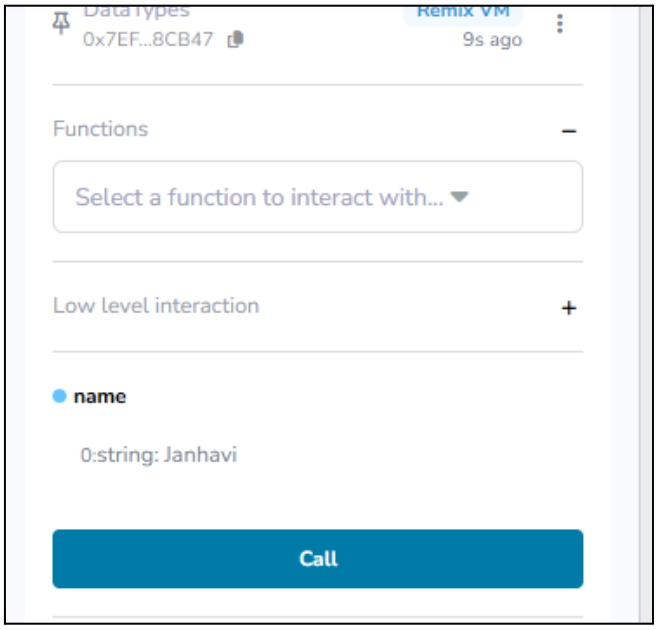
```
1   // SPDX-License-Identifier: MIT
2   pragma solidity ^0.8.18;
3
4   contract BasicContract {
5       string public name = "Janhavi";
6
7       function getName() public view returns (string memory) {    infinite gas
8           return name;
9       }
10  }
```



Tutorial no. 2 Primitive Data Types and Variables

```
contract DataTypes {
    uint public age = 21;
    int public number = -10;
    bool public status = true;
    address public owner;
    string public name = "Janhavi";

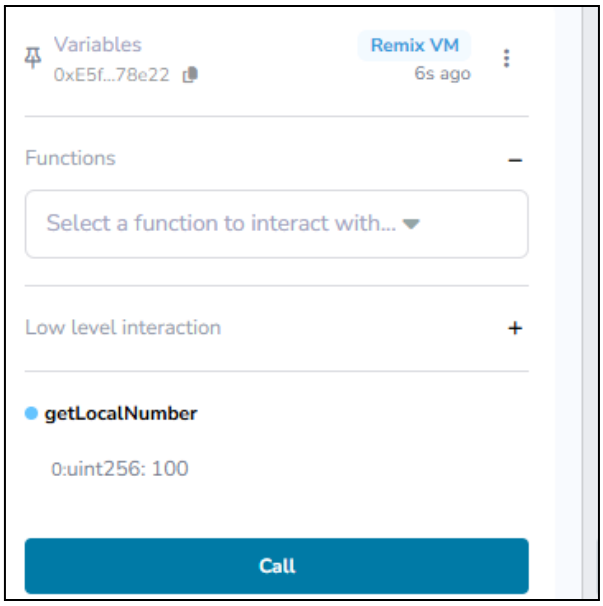
    constructor() {    infinite gas 194800 gas
        owner = msg.sender;
    }
}
```



Tutorial no. 3 State, Local and Global Variables

```
contract Variables {
    uint public stateNumber = 30;

    function getLocalNumber() public pure returns (uint) {    322 gas
        uint localNumber = 100;
        return localNumber;
    }
}
```

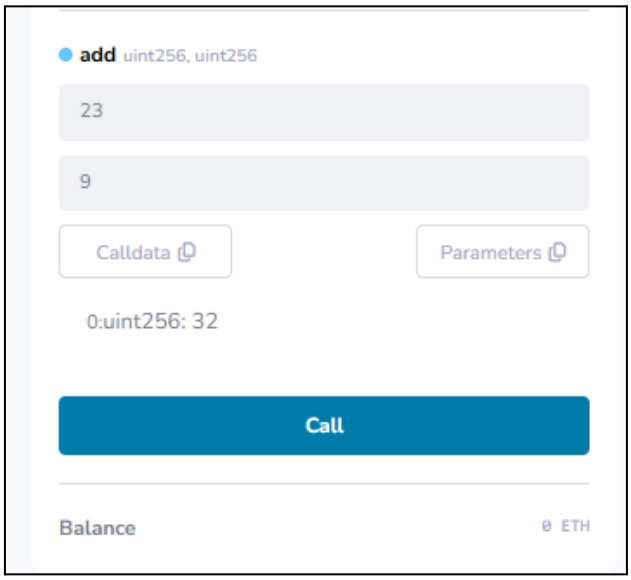

Variables
0xE5f...78e22
Remix VM
6s ago
Functions
Select a function to interact with...
Low level interaction
getLocalNumber
0:uint256: 100
Call

Tutorial no. 4 Functions – Pure and View

```
contract Functions {
    uint public number = 10;

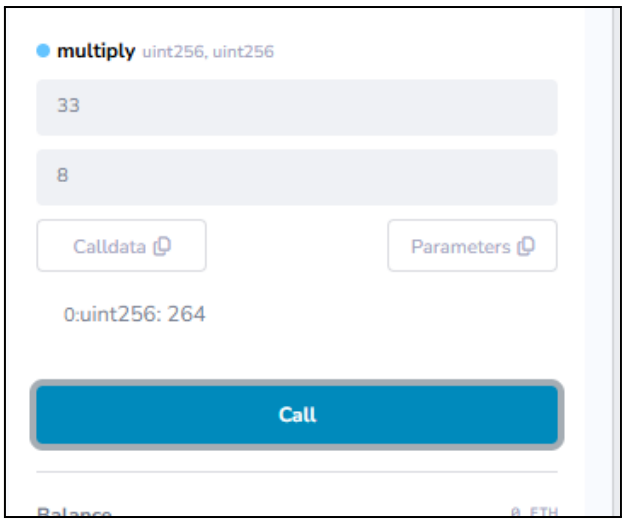
    function getNumber() public view returns (uint) {    2453 gas
        return number;
    }

    function add(uint a, uint b) public pure returns (uint) {    infinite gas
        return a + b;
    }
}
```



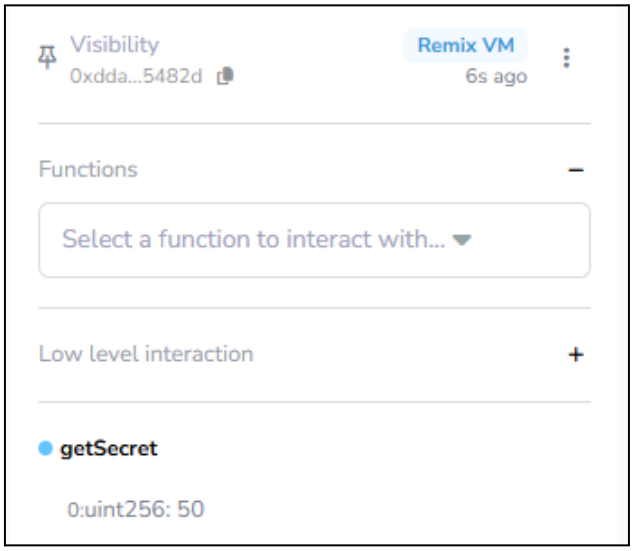
Tutorial no. 5 Function Inputs and Outputs

```
contract InputOutput {
    function multiply(uint a, uint b) public pure returns (uint) {    infinite gas
        return a * b;
    }
}
```



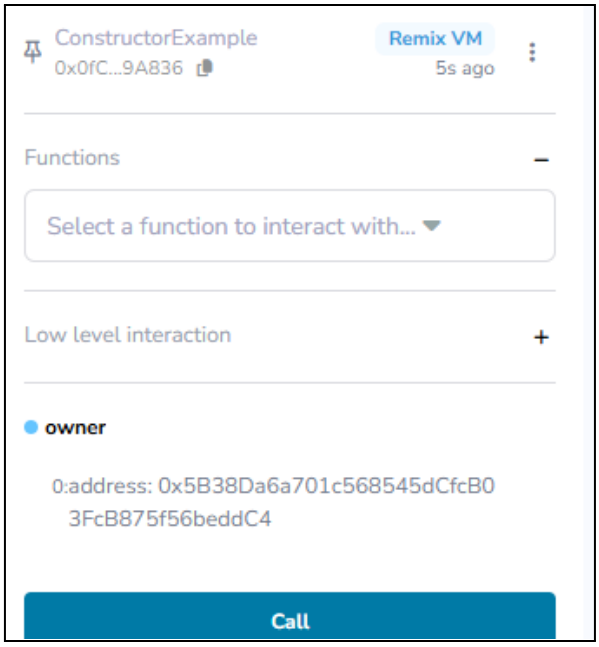
Tutorial no. 6 Function Visibility

```
1   contract Visibility {
2       uint private secret = 50;
3
4       function getSecret() public view returns (uint) {    2409 gas
5           return secret;
6       }
7   }
```



Tutorial no. 7 Constructors

```
1   contract ConstructorExample {
2       address public owner;
3
4       constructor() {    72361 gas 48000 gas
5           owner = msg.sender;
6       }
7   }
```


ConstructorExample
0x0fC...9A836
Remix VM
5s ago
Functions
Select a function to interact with...
Low level interaction
owner
0:address: 0x5B38Da6a701c568545dCfcB0
3FcB875f56beddC4
Call

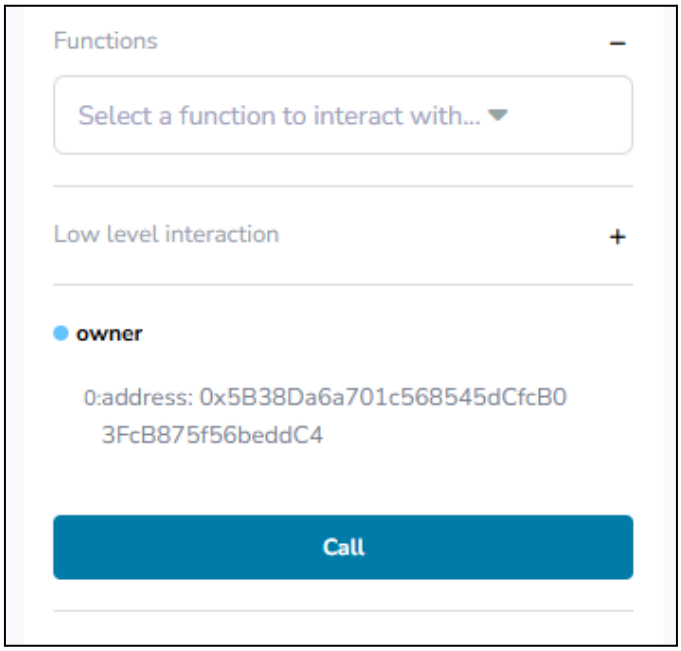
Tutorial no. 8 Function Modifiers

```
contract ModifierExample {
    address public owner;

    constructor() {    166252 gas 141800 gas
        owner = msg.sender;
    }

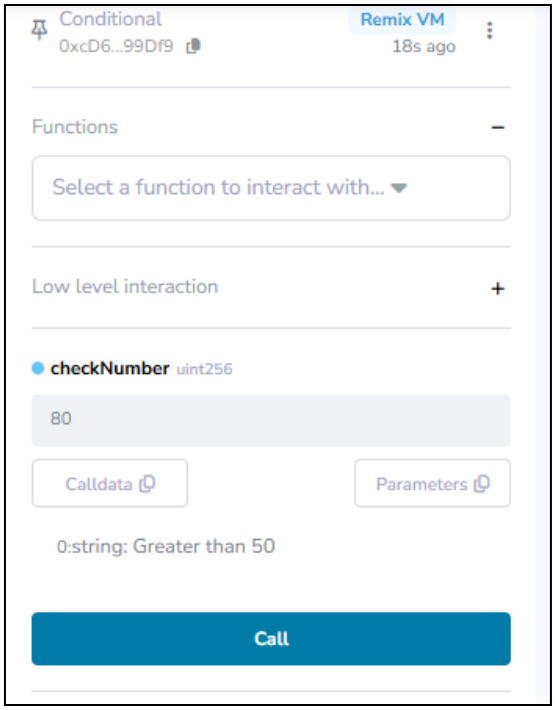
    modifier onlyOwner() {
        require(msg.sender == owner, "Not the owner");
        _;
    }

    function changeOwner(address newOwner) public onlyOwner {    26882 gas
        owner = newOwner;
    }
}
```



Tutorial no. 9 Conditional Statements – If-Else

```
contract Conditional {
    function checkNumber(uint number) public pure returns (string memory) {    infi
        if (number > 50) {
            return "Greater than 50";
        } else {
            return "50 or less";
        }
    }
}
```

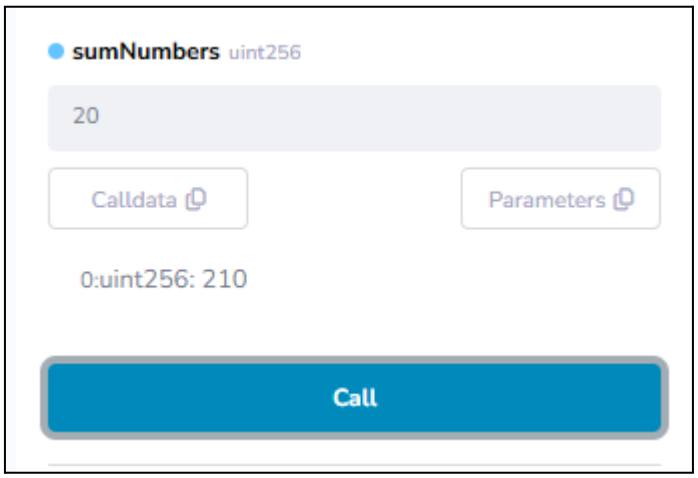

Conditional
0xcD6...99Df9
Remix VM
18s ago
Functions
Select a function to interact with...
Low level interaction
checkNumber uint256
80
Calldata
Parameters
0:string: Greater than 50
Call

Tutorial no. 10 Loops

```
contract LoopExample {
    function sumNumbers(uint n) public pure returns (uint) {    infinite gas
        uint sum = 0;

        for (uint i = 1; i <= n; i++) {
            sum += i;
        }

        return sum;
    }
}
```

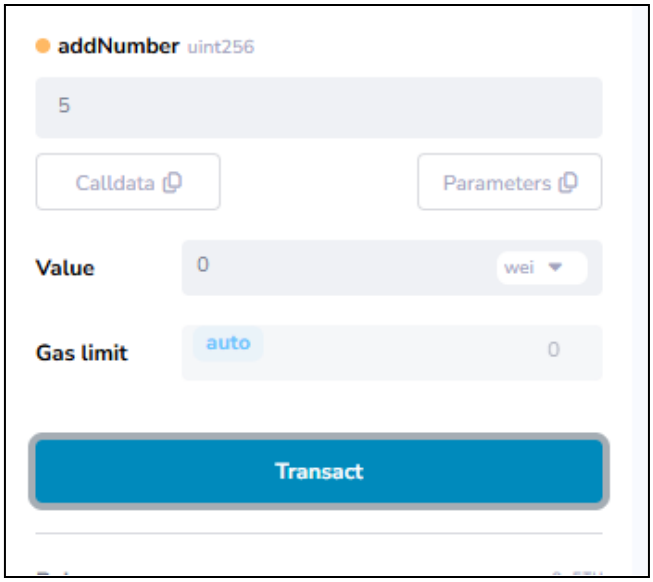


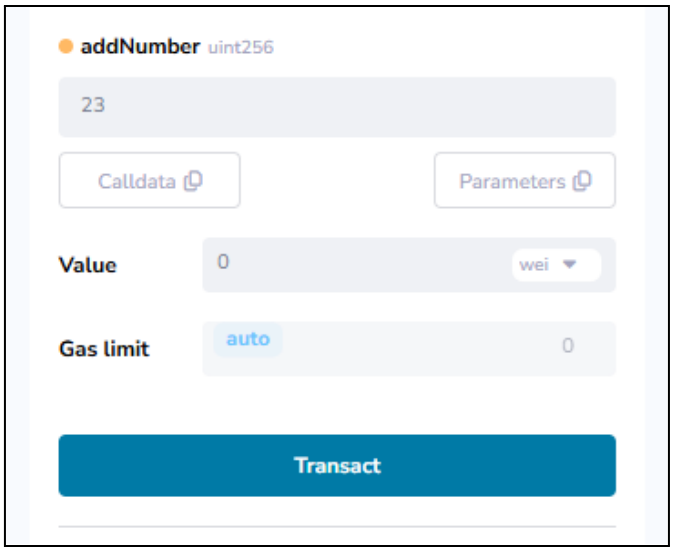
Tutorial no. 11 Arrays

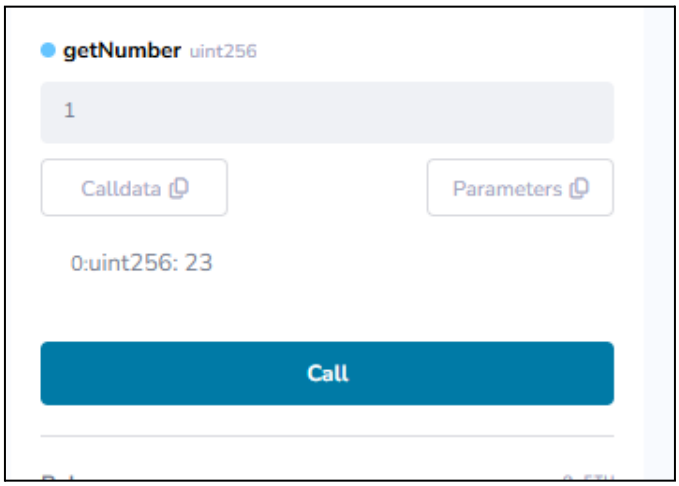
```
contract ArrayExample {
    uint[] public numbers;

    function addNumber(uint number) public {    46842 gas
        numbers.push(number);
    }

    function getNumber(uint index) public view returns (uint) {    infinite gas
        return numbers[index];
    }
}
```


addNumber uint256
5
Calldata
Parameters
Value
0
wei
Gas limit
auto
0
Transact

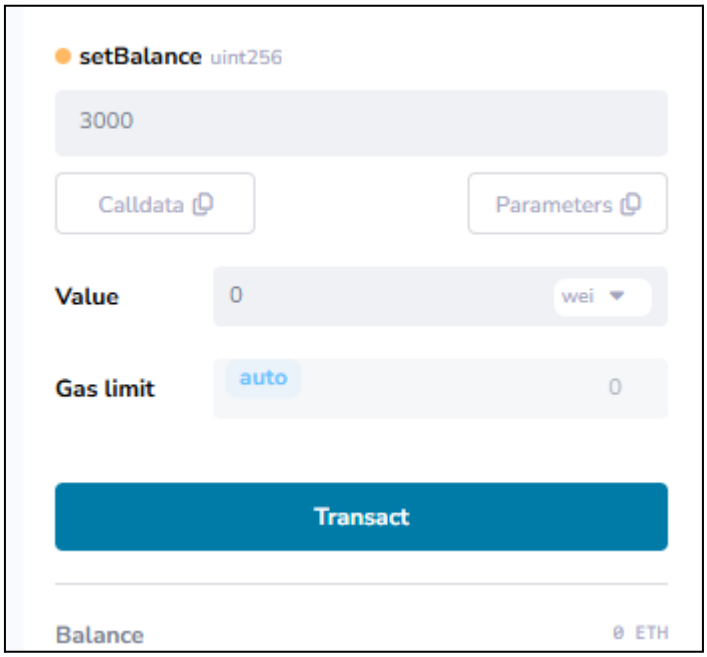

addNumber uint256
23
Calldata
Parameters
Value
0
wei
Gas limit
auto
0
Transact


getNumber uint256
1
Calldata
Parameters
0:uint256: 23
Call

Tutorial no. 12 Mappings

```
contract MappingExample {
    mapping(address => uint) public balances;

    function setBalance(uint amount) public {    22601 gas
        balances[msg.sender] = amount;
    }
}
```

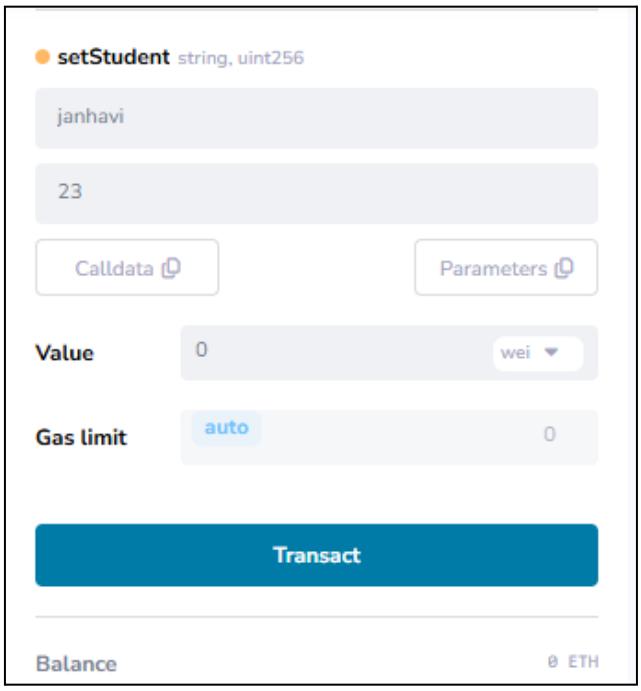


Tutorial no. 13 Structures (struct)

```
contract StructExample {
    struct Student {
        string name;
        uint rollNo;
    }

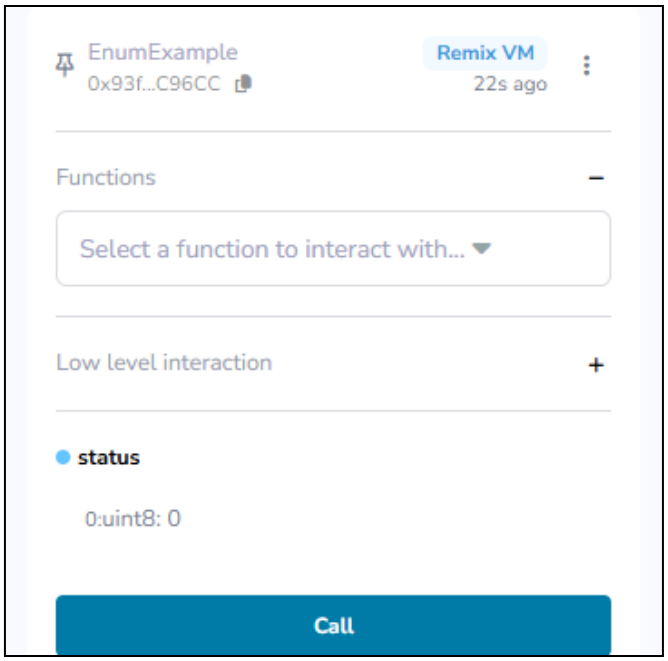
    Student public student;

    function setStudent(string memory _name, uint _rollNo) public {    infinite gas
        student = Student(_name, _rollNo);
    }
}
```



Tutorial no. 14 Enumerations (Enums)

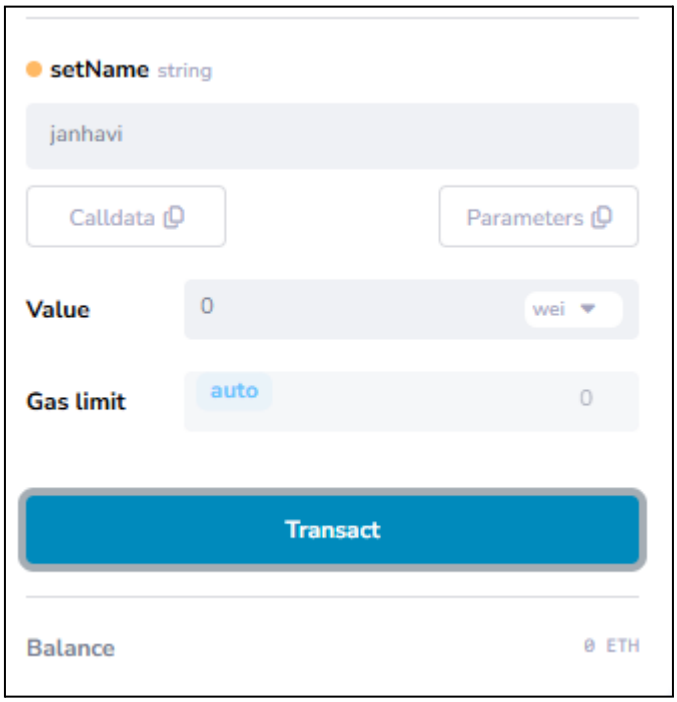
```
1   contract EnumExample {
2       enum Status {
3           Pending,
4           Active,
5           Completed
6       }
7
8       Status public status;
9
10      function setActive() public {    24450 gas
11          status = Status.Active;
12      }
13
14      function setCompleted() public {    24428 gas
15          status = Status.Completed;
16      }
17  }
```



Tutorial no. 15 Data Locations – Storage, Memory and Calldata

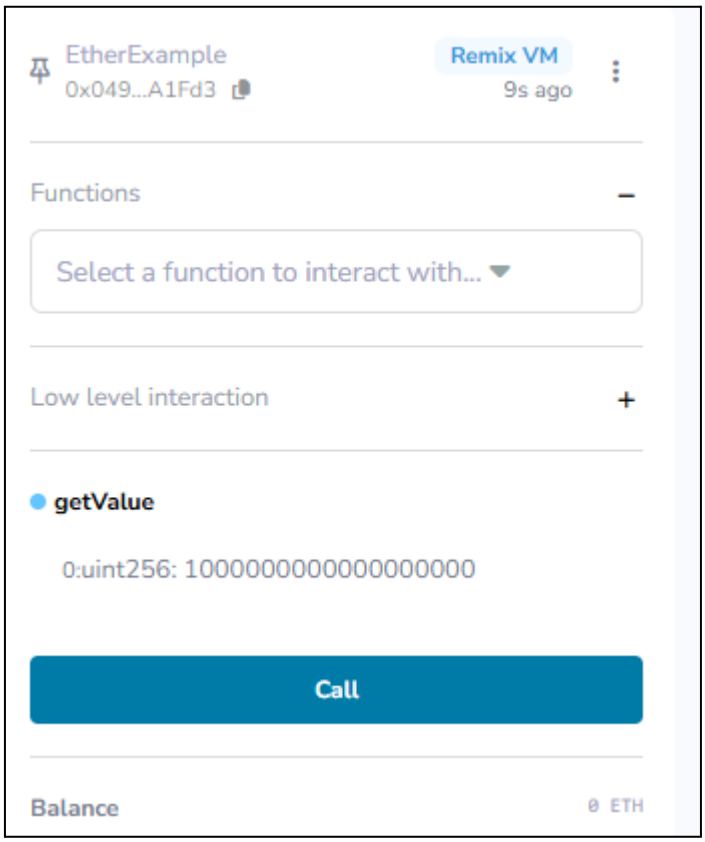
```
contract DataLocation {
    string public name;

    function setName(string memory _name) public {
        name = _name;
    }
}
```

Tutorial no. 16 Ether and Wei

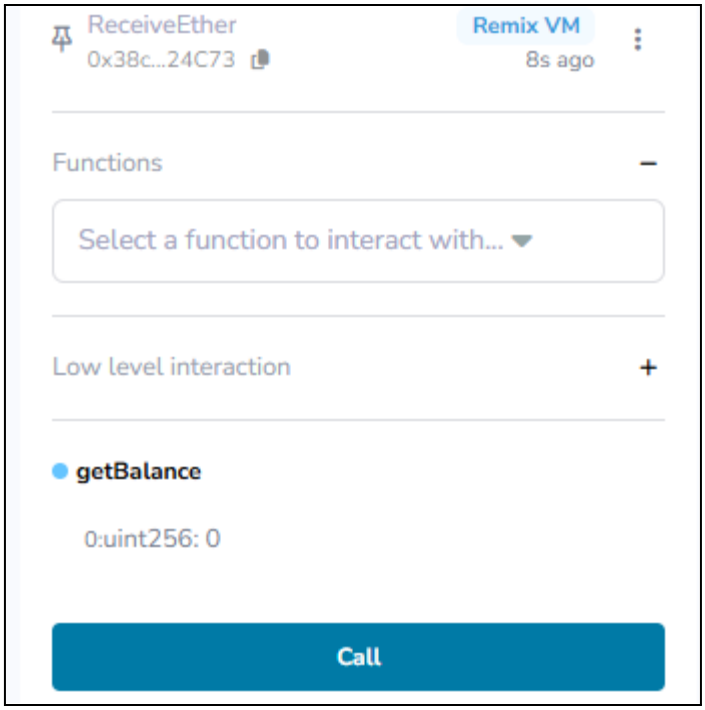
```
contract EtherExample {
    function getValue() public pure returns (uint) {    310 gas
        return 1 ether;
    }
}
```



Tutorial no. 17 Receiving and Sending Ether

```
contract ReceiveEther {
    receive() external payable {}    undefined gas

    function getBalance() public view returns (uint) {    312 gas
        return address(this).balance;
    }
}
```


ReceiveEther
0x38c...24C73
Remix VM
8s ago
Functions
Select a function to interact with...
Low level interaction
getBalance
0:uint256: 0
Call

Tutorial no. 18 Gas and Gas Price

```
contract ReceiveEther {
    receive() external payable {}    undefined gas

    function getBalance() public view returns (uint) {    312 gas
        return address(this).balance;
    }
}
```

```
to                      ReceiveEther.getBalance() 0x38cB7800C3Fddb8dda074C1c650A155154924C73
execution cost          312 gas (Cost only applies when called by a contract)
input                   0x120...65fe0
output                  0x0000000000000000000000000000000000000000000000000000000000000000
decoded input           {}
decoded output          {
                            "0": "uint256: 0"
                        }
logs                    []
```

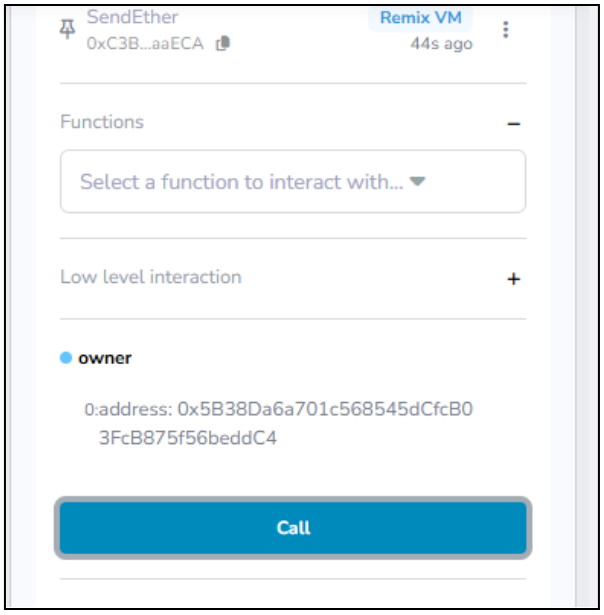
Tutorial no. 19 Blockchain Transactions and Transaction Execution

```
contract SendEther {
    address payable public owner;

    constructor() {    180064 gas 155600 gas
        owner = payable(msg.sender);
    }

    receive() external payable {}    undefined gas

    function withdraw(uint amount) public {    infinite gas
        require(msg.sender == owner, "Only owner");
        owner.transfer(amount);
    }
}
```


SendEther
0xC3B...aaECA
Remix VM
44s ago
Functions
Select a function to interact with...
Low level interaction
owner
0:address: 0x5B38Da6a701c568545dCfcB0
3FcB875f56beddC4
Call

Conclusion:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.